SQL · ESTRUCTURA BÁSICA DE CONSULTAS

Clase 1 · Diseño de Base de Datos

Ingeniería en Informática · 2.º año · Universidad Católica de Córdoba

Agenda

0. **Apertura — qué necesitás saber antes**
1. **SQL como lenguaje — declarativo, componentes, estándares**
2. **DDL básico — tipos, `CREATE TABLE`, PK y FK**
3. **El esquema SiGUE — la base que vamos a usar todo el cuatrimestre**
4. **`SELECT`, `FROM`, `WHERE` — el corazón de SQL**
5. **Cadenas, `ORDER BY`, `DISTINCT`, alias**
6. **JOINs — combinar datos de varias tablas**

Al final vas a poder leer y escribir consultas que crucen 2 o 3 tablas con filtros y orden.

¿Qué necesitás saber?

1 · MODELO RELACIONAL

Tablas (relaciones), filas (tuplas), columnas (atributos). Cada tabla representa una entidad o un vínculo entre entidades.

2 · CLAVES PRIMARIA Y FORÁNEA

La PK identifica unívocamente la fila. La FK referencia la PK de otra tabla y materializa la relación.

3 · ENTORNO DE TRABAJO

MySQL 8 + MySQL Workbench. La base `sigue` ya está cargada en el campus con datos de ejemplo.

UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS
70 AÑOS

01

SQL como lenguaje

Qué es, de dónde viene, por qué es declarativo

SQL: *Structured Query Language*

Lenguaje estándar para interactuar con bases de datos relacionales. Nació en IBM a inicios de los 70 en el proyecto **System R**, originalmente llamado *Sequel*.

Estandarizado por ANSI/ISO desde 1986: SQL-86 → SQL-92 → SQL:1999 → SQL:2003 → SQL:2008 → SQL:2016.

> ⚠ **OJO**
> La norma existe, pero **cada motor** (MySQL, PostgreSQL, Oracle, SQL Server) implementa subconjuntos diferentes y agrega extensiones propias. Una consulta que corre en uno puede fallar en otro. Siempre conviene leer el manual del motor.

SQL no es solo consultar

DDL

Data Definition. Crea, modifica y borra estructuras: tablas, índices, vistas.

CREATE · ALTER · DROP

DML

Data Manipulation. Inserta, modifica, borra y consulta datos.

SELECT · INSERT · UPDATE · DELETE

DCL

Data Control. Maneja permisos sobre objetos de la base.

GRANT · REVOKE

TCL

Transaction Control. Marca el inicio y el fin de unidades atómicas.

COMMIT · ROLLBACK

En esta clase tocamos DDL y arrancamos con la parte de consulta de DML. El resto viene en las próximas clases y al final.

Declarativo vs imperativo

Misma tarea: *traer los alumnos cuyo apellido empieza con "G"*.

IMPERATIVO (CÓMO)

```
# Pseudocódigo
resultado = []
for a in alumnos:
    if a.apellido.startswith("G"):
        resultado.append(a)
return resultado
```

Le decís al motor cada paso: recorré, filtrá, guardá, devolvé.

DECLARATIVO (QUÉ)

```
-- SQL
SELECT *
FROM alumnos
WHERE apellido LIKE 'G%';
```

Le decís qué resultado querés. El motor decide cómo conseguirlo.

¿Por qué importa que sea declarativo?

1 · EL MOTOR OPTIMIZA POR VOS

Decide en qué orden recorrer las tablas, qué índice usar, si paralelizar. Si el "cómo" estuviera fijo, no podría mejorarlo.

2 · LA CONSULTA NO CAMBIA SI CAMBIA EL MOTOR

La misma `SELECT` corre en MySQL, PostgreSQL u Oracle (con variantes mínimas). El plan de ejecución cambia, tu código no.

3 · TE CONCENTRÁS EN EL PROBLEMA, NO EN LA MECÁNICA

"Quiero los alumnos sin notas cargadas" se escribe casi como en castellano. No tenés que pensar en bucles, índices ni estructuras.

✓ TIP

El motor de la BD tiene un componente llamado **optimizador de consultas** que toma tu `SELECT` y arma un *plan de ejecución*. Lo vemos al final del cuatrimestre.

02

DDL — Definición de datos

Crear y modificar la estructura de la base

Tipos de datos más usados

Tipo	Significado y uso
`INT`	Entero. *id_alumno*, *cupo*, *anio*.
`DECIMAL(p,d)`	Coma fija con *p* dígitos totales y *d* decimales. Ideal para dinero y notas: `DECIMAL(4,2)` permite `10.00` exacto.
`VARCHAR(n)`	Cadena de longitud variable, hasta *n*. *nombre*, *apellido*, *email*.
`CHAR(n)`	Cadena de longitud **fija**. Si guardás `'AB'` en `CHAR(5)`, se rellena con espacios. Usar solo cuando el largo es siempre el mismo.
`DATE` · `DATETIME`	Fecha (sin hora) y fecha + hora. *fecha_inscripcion*, *fecha_nacimiento*.
`TINYINT(1)`	En MySQL se usa como booleano. *presente*, *aprobado*, *es_obligatoria*.
`ENUM(...)`	Lista cerrada de valores. *estado* ∈ {'activa','baja','egresada'}.

✓ **TIP**

Por defecto, todos los tipos admiten `NULL` (ausencia de valor). Si no querés nulos, lo declarás con `NOT NULL`.

CREATE TABLE: forma general

```
CREATE TABLE nombre_tabla (
    columna_1 tipo  [restricciones],
    columna_2 tipo  [restricciones],
    ...
    PRIMARY KEY (col_pk),
    FOREIGN KEY (col_fk) REFERENCES otra_tabla(col_pk)
);
```

Cada columna se define con **nombre + tipo + restricciones opcionales**. Al final se declaran las restricciones de tabla: PK, FK y constraints compuestos.

Ejemplo: la tabla `alumnos`

```sql
CREATE TABLE alumnos (
    id_alumno         INT           NOT NULL AUTO_INCREMENT,
    nombre            VARCHAR(100)  NOT NULL,
    apellido          VARCHAR(100)  NOT NULL,
    dni               VARCHAR(20)   NOT NULL,
    fecha_nacimiento  DATE,
    email             VARCHAR(150),
    telefono          VARCHAR(50),
    fecha_ingreso     DATE          NOT NULL,
    PRIMARY KEY (id_alumno),
    UNIQUE (dni)
);
```

✓ TIP MYSQL

`AUTO_INCREMENT` hace que el motor genere automáticamente el siguiente `id_alumno` cuando insertás sin especificarlo.

PRIMARY KEY y FOREIGN KEY

PRIMARY KEY

Identifica unívocamente cada fila. Implica **NOT NULL** + **UNIQUE**. Una tabla tiene *como máximo* una PK (puede ser compuesta por varias columnas).

FOREIGN KEY

Vincula esta tabla con otra. Los valores de la FK deben existir como PK en la tabla referenciada (o ser `NULL`). Es la **integridad referencial**.

```sql
-- En SiGUE: una comisión pertenece a una asignatura y la dicta un docente
CREATE TABLE comisiones (
    id_comision    INT  NOT NULL AUTO_INCREMENT,
    id_asignatura  INT  NOT NULL,
    id_docente     INT  NOT NULL,
    anio           INT  NOT NULL,
    PRIMARY KEY (id_comision),
    FOREIGN KEY (id_asignatura) REFERENCES asignaturas(id_asignatura),
    FOREIGN KEY (id_docente)    REFERENCES docentes(id_docente)
);
```

Modificar una tabla existente

```sql
-- Agregar una columna (las filas existentes quedan con NULL)
ALTER TABLE alumnos ADD direccion VARCHAR(200);

-- Quitar una columna
ALTER TABLE alumnos DROP COLUMN telefono;

-- Cambiar el tipo o las restricciones
ALTER TABLE alumnos MODIFY email VARCHAR(200) NOT NULL;

-- Agregar una restricción a posteriori
ALTER TABLE alumnos ADD CONSTRAINT uq_email UNIQUE (email);
```

⚠ **OJO**

En tablas con muchos datos, un `ALTER` puede ser **caro**: el motor a veces reconstruye la tabla entera. Antes de hacerlo en producción, mirá la documentación.

Tres formas de borrar (y son distintas)

Comando	Categoría	Qué hace	Recuperable con
`DROP TABLE`	DDL	Borra la tabla y su definición. **No queda nada**: hay que volver a hacer `CREATE TABLE`.	No
`TRUNCATE TABLE`	DDL	Vacía la tabla, conserva la estructura. Es muy rápido. Resetea `AUTO_INCREMENT`.	No
`DELETE FROM`	DML	Borra filas, una por una, según un `WHERE`. Si no hay `WHERE`, borra todas pero con costo.	Sí

⚠ **OJO**

`DELETE FROM tabla;` sin `WHERE` borra **todas** las filas. Si nunca lo hiciste antes, te va a sorprender la primera vez.

03

Esquema SiGUE

La base que vamos a usar todo el cuatrimestre

SiGUE — núcleo de 8 tablas

Sistema de Gestión Universitaria Estudiantil. La base completa tiene ~16 tablas; en clase trabajamos con este subset estable.

alumnos

- **id_alumno**
- nombre
- apellido
- dni
- fecha_nacimiento
- email
- fecha_ingreso

inscripcion_alumno_comision

- **id_inscripcion**
- *id_alumno*
- *id_comision*
- fecha_inscripcion
- estado

comisiones

- **id_comision**
- *id_asignatura*
- *id_docente*
- anio
- cuatrimestre
- cupo

asignaturas

- **id_asignatura**
- nombre
- creditos
- carga_horaria
- es_obligatoria
- *id_departamento*

notas

- **id_nota**
- *id_examen*
- *id_alumno*
- nota
- aprobado

docentes

- **id_docente**
- nombre
- apellido
- email
- salario_base
- *id_departamento*

departamentos

- **id_departamento**
- nombre
- presupuesto
- fecha_creacion

carreras

- **id_carrera**
- nombre
- titulo_otorgado
- nivel
- duracion_anios
- *id_departamento*

subrayado = clave primaria · *cursiva* = clave foránea

04

Consultas: SELECT, FROM, WHERE

El corazón de SQL

SELECT: estructura general

```
SELECT   columnas              -- qué quiero ver
FROM     tabla_o_tablas        -- de dónde lo saco
WHERE    predicado             -- qué filas me interesan
ORDER BY columnas              -- en qué orden las quiero;
```

Las tres cláusulas **fundamentales** son `SELECT`, `FROM` y `WHERE`. Si omitís `WHERE`, el predicado por defecto es *verdadero* y pasan todas las filas.

Traer todo, traer algo

```sql
-- Todas las columnas, todas las filas
SELECT * FROM alumnos;

-- Solo las columnas que necesito
SELECT apellido, nombre, email
FROM alumnos;

-- Calculo en la consulta (no modifica la tabla)
SELECT apellido, salario_base * 1.10 AS salario_con_aumento
FROM docentes;
```

✓ **TIP**

`SELECT *` es cómodo para explorar pero **en producción se evita**: si después se agrega una columna a la tabla, tu query empieza a traer datos que no esperabas.

Filtrar filas con WHERE

Acepta operadores de comparación (=, <>, <, ⩽, >, ⩾) y conectores lógicos (`AND`, `OR`, `NOT`).

```sql
-- Comisiones del segundo cuatrimestre de 2025
SELECT id_comision, id_asignatura, cupo
FROM comisiones
WHERE anio = 2025 AND cuatrimestre = 2;

-- Notas aprobadas (≥ 4)
SELECT id_alumno, nota
FROM notas
WHERE nota ⩾ 4;

-- Docentes que NO son del departamento 3
SELECT nombre, apellido
FROM docentes
WHERE id_departamento <> 3;
```

Operadores especiales del WHERE

```sql
-- BETWEEN: rango inclusivo (equivale a ≥ AND ≤)
SELECT id_alumno, nota
FROM notas
WHERE nota BETWEEN 7 AND 10;

-- IN: pertenencia a un conjunto literal
SELECT nombre, apellido
FROM docentes
WHERE id_departamento IN (1, 3, 5);

-- LIKE: patrón con comodines
SELECT apellido
FROM alumnos
WHERE apellido LIKE 'G%';
```

✓ TIP

Las tres formas tienen su versión negada: `NOT BETWEEN`, `NOT IN`, `NOT LIKE`.

Orden de escritura vs orden de ejecución

Lo escribís así:

`SELECT` → `FROM` → `WHERE` → `ORDER BY`

El motor lo ejecuta así:

`FROM` → `WHERE` → `SELECT` → `ORDER BY`

¿POR QUÉ IMPORTA?

El motor primero **arma el conjunto de filas** (FROM + WHERE), y recién después decide qué columnas mostrar (SELECT). Por eso un alias creado en `SELECT` **no se puede usar dentro del WHERE**: cuando el WHERE corre, ese alias todavía no existe.

En la próxima clase ampliamos el orden con GROUP BY y HAVING.

05

Cadenas, ORDER BY, DISTINCT, alias

Ajustes finales al resultado

Operaciones con cadenas

Las cadenas van siempre entre **comillas simples**. Para una comilla literal, se duplica: `'O''Brien'`.

```sql
-- LIKE con comodines: % = cualquier cadena, _ = un solo caracter
SELECT nombre FROM asignaturas WHERE nombre LIKE 'Base%';     -- empieza con "Base"
SELECT nombre FROM asignaturas WHERE nombre LIKE '%Datos%';    -- contiene "Datos"
SELECT dni    FROM alumnos     WHERE dni    LIKE '__.___.___';   -- exactamente 11 caracteres con puntos

-- Funciones útiles
SELECT UPPER(apellido), LOWER(email), LENGTH(nombre)
FROM alumnos;

-- Concatenación en MySQL: CONCAT (el || estándar no funciona por defecto)
SELECT CONCAT(apellido, ', ', nombre) AS nombre_completo
FROM alumnos;
```

⚠ **OJO**

En MySQL, por defecto las comparaciones de texto son **case-insensitive**: `'PEREZ' = 'perez'`. En PostgreSQL u Oracle es al revés.

ORDER BY: ordenar el resultado

```sql
-- Por defecto, ascendente (ASC)
SELECT apellido, nombre
FROM alumnos
ORDER BY apellido;

-- Descendente y por varias columnas
SELECT apellido, nombre, fecha_ingreso
FROM alumnos
ORDER BY fecha_ingreso DESC, apellido ASC;

-- Ordenar por una expresión calculada
SELECT nombre, salario_base * 1.10 AS nuevo_sueldo
FROM docentes
ORDER BY nuevo_sueldo DESC;
```

Sin `ORDER BY`, el motor no garantiza ningún orden. Aunque parezca consistente al probar, puede cambiar con datos distintos.

DISTINCT: quitar duplicados

SQL **conserva duplicados** por defecto. Para forzar valores únicos, se usa `DISTINCT`.

```sql
-- ¿Qué departamentos tienen al menos un docente? (con duplicados)
SELECT id_departamento FROM docentes;        -- repite filas

-- Sin duplicados
SELECT DISTINCT id_departamento FROM docentes;

-- DISTINCT aplica al CONJUNTO de columnas, no a cada columna por separado
SELECT DISTINCT anio, cuatrimestre FROM comisiones;
```

⚠ OJO

`DISTINCT` es **caro**: el motor tiene que comparar todas las filas para detectar duplicados. Usalo solo cuando lo necesitás de verdad.

Renombrar con AS

```sql
-- Renombrar columnas del resultado (útil para expresiones)
SELECT apellido AS apellido_alumno,
       YEAR(fecha_ingreso) AS anio_ingreso
FROM alumnos;

-- Acortar el nombre de una tabla en la consulta
SELECT a.apellido, a.email
FROM alumnos AS a
WHERE a.fecha_ingreso ⩾ '2024-01-01';
```

✓ TIP

Cuando trabajás con varias tablas (los `JOIN` que vienen ahora), los alias cortos hacen que la consulta sea legible. **La palabra `AS` es opcional**: `FROM alumnos a` también funciona.

Pausa: probalo vos

Consigna: traer los *apellidos únicos* de los alumnos cuyo email termine en `@ucc.edu.ar`, ordenados alfabéticamente.

```sql
SELECT DISTINCT apellido
FROM alumnos
WHERE email LIKE '%@ucc.edu.ar'
ORDER BY apellido;
```

¿POR QUÉ ESTE ORDEN DE CLÁUSULAS FUNCIONA?

El motor primero hace `FROM alumnos`, después aplica el `WHERE` (descarta filas), después proyecta `apellido`, después aplica `DISTINCT` y al final ordena.

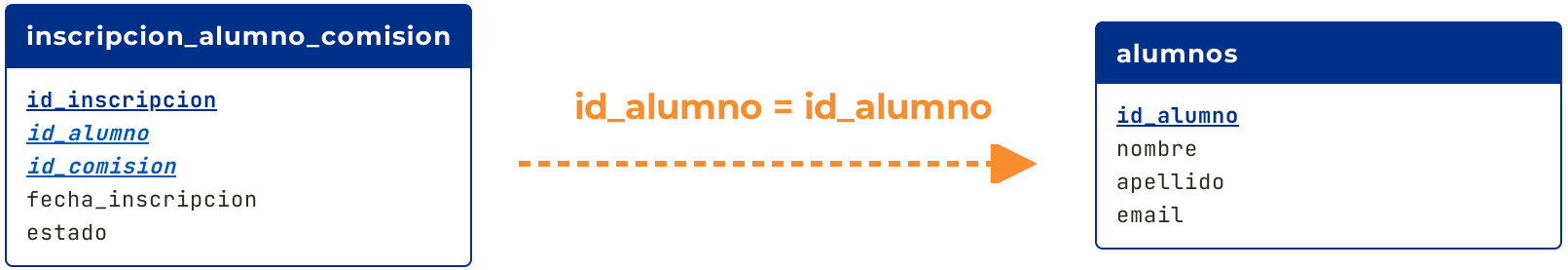
06

JOINs básicos

Combinar datos de varias tablas

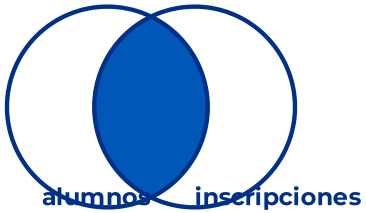
El problema

En `inscripcion_alumno_comision` tenés `id_alumno` y `id_comision`. **No** tenés el nombre del alumno ni la asignatura. ¿Cómo los traés?



`JOIN` combina filas de dos tablas según una **condición de relación** (típicamente PK = FK).

INNER JOIN: solo lo que matchea en ambas




```sql
-- Apellido + estado de inscripción, comisión 12
SELECT a.apellido, a.nombre, i.estado
FROM alumnos AS a
INNER JOIN inscripcion_alumno_comision AS i
  ON a.id_alumno = i.id_alumno
WHERE i.id_comision = 12;
```

CÓMO SE LEE

Por cada fila de `alumnos`, busco las filas de `inscripcion_alumno_comision` donde `id_alumno` coincida. Si no hay coincidencia, **se descarta**. El resultado solo contiene alumnos que están inscriptos.

WHERE vs ON: ¿dónde poner la condición?

CONDICIÓN DE UNIÓN → ON

Cómo se relacionan las dos tablas. Casi siempre `PK = FK`.

```
FROM alumnos a
INNER JOIN notas n
  ON a.id_alumno = n.id_alumno
```

FILTRO DEL RESULTADO → WHERE

Qué filas del resultado final me interesan.

```
FROM alumnos a
INNER JOIN notas n
  ON a.id_alumno = n.id_alumno
WHERE n.nota ⩾ 7
```

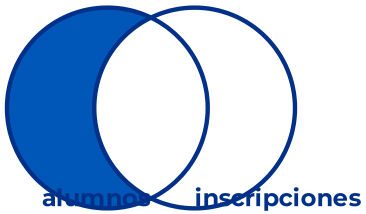
EN INNER JOIN, LA DIFERENCIA ES ESTILÍSTICA

El resultado es el mismo si pones `n.nota ⩾ 7` en el `ON` o en el `WHERE`. Pero **en LEFT/RIGHT JOIN sí cambia**: lo vemos en dos slides.

✓ CONVENCIÓN

En `ON` va solo la *condición de relación entre tablas*. Cualquier filtro adicional va en `WHERE`. Hace la consulta más legible.

LEFT JOIN: todo lo de la izquierda




```sql
-- Todos los alumnos, con sus inscripciones (si las tienen)
SELECT a.id_alumno, a.apellido, i.id_comision, i.estado
FROM alumnos AS a
LEFT JOIN inscripcion_alumno_comision AS i
  ON a.id_alumno = i.id_alumno;
```

Si un alumno no tiene inscripciones, las columnas de la derecha aparecen como `NULL`.

✓ TIP · CASO DE USO CLÁSICO

Útil para encontrar **"huérfanos"**: alumnos sin inscribirse, asignaturas sin comisión, docentes sin asignación. Combinalo con `WHERE columna_derecha IS NULL` para listarlos.

RIGHT JOIN: el espejo del LEFT




```sql
-- Todas las asignaturas, con sus comisiones (si tienen)
SELECT c.id_comision, c.anio, asg.nombre
FROM comisiones AS c
RIGHT JOIN asignaturas AS asg
  ON c.id_asignatura = asg.id_asignatura;
```

Conserva todas las filas de la tabla **derecha**; las columnas de la izquierda quedan `NULL` cuando no hay match.

✓ CONVENCIÓN

En la práctica casi nadie usa `RIGHT JOIN`: se invierte el orden de las tablas y se usa `LEFT JOIN`. Es más legible.

JOIN encadenado: 3+ tablas

```sql
-- Listado: alumno · asignatura · docente · estado de inscripción
SELECT a.apellido     AS alumno,
       asg.nombre     AS asignatura,
       d.apellido     AS docente,
       i.estado
FROM inscripcion_alumno_comision AS i
INNER JOIN alumnos     AS a   ON i.id_alumno    = a.id_alumno
INNER JOIN comisiones  AS c   ON i.id_comision  = c.id_comision
INNER JOIN asignaturas AS asg ON c.id_asignatura = asg.id_asignatura
INNER JOIN docentes    AS d   ON c.id_docente   = d.id_docente
ORDER BY a.apellido, asg.nombre;
```

✓ TIP

Cuando hay 3+ tablas, los **alias cortos** (`a`, `c`, `asg`, `d`) hacen toda la diferencia. Cada `ON` se lee como una frase: "uno alumnos con inscripciones por id_alumno".

Pausa: probalo vos

Consigna: listar todos los *docentes* con el nombre del *departamento* al que pertenecen, ordenados por departamento y dentro de cada departamento por apellido.

```sql
SELECT dep.nombre AS departamento,
       d.apellido,
       d.nombre
FROM docentes AS d
INNER JOIN departamentos AS dep
  ON d.id_departamento = dep.id_departamento
ORDER BY dep.nombre, d.apellido;
```

VARIANTE

¿Y si quisieras incluir docentes **sin departamento asignado**? Cambiás `INNER JOIN` por `LEFT JOIN` y aparecen con `departamento = NULL`.

Lo que vimos hoy

- **SQL es declarativo**: decís qué querés, no cómo conseguirlo.
- **DDL**: `CREATE / ALTER / DROP / TRUNCATE`, tipos básicos, PK y FK.
- **SELECT/FROM/WHERE** con operadores `BETWEEN`, `IN`, `LIKE`.
- **Refinamientos**: `ORDER BY`, `DISTINCT`, alias `AS`.
- **JOINs básicos**: `INNER`, `LEFT`, `RIGHT`, encadenados.
- **Orden de ejecución**: FROM → WHERE → SELECT → ORDER BY.

PRÓXIMA CLASE · AGREGACIÓN, CONJUNTOS Y NULL

Funciones de agregación (`COUNT`, `SUM`, `AVG`…), `GROUP BY`, `HAVING`, operaciones de conjunto (`UNION`, `INTERSECT`, `EXCEPT`) y la lógica trivaluada de `NULL`.

Práctico de la semana: ejercicios sobre el esquema SiGUE en Moodle.